Xenium spatial transcriptomics viewer — conversation transcript

Exported from a Claude Code session. User messages and assistant replies are verbatim; each tool call is shown as a one-line note (▸), and tool outputs, intermediate reasoning and system messages are omitted.

User · 2026-10-14 02:02 UTC

My goal is to create an interactive viewer for Xenium spatial transcriptomics data, with cells overlaid on the user's choice of H&E image or Xenium DAPI image. Ideally, this tool will also be able to label cells by their cluster label or the cluster label as transferred from a single-nucleus RNAseq reference dataset. Additionally, several varieties of a "domain" as defined from transferred labels and knn smoothing are included in the data. Ideally, the viewer would display this domain as a polygonal outline demarcating its boundaries, rather than colorizing cells based on their domain assignment. (That way, the user can e.g., see cells colorized by cluster but also see the computationally-defined boundaries of an area of interest).

Finally, the tool should have an option to colorize spots by the normalized expression (the `Assay logcounts` in the SpatialExperiment file) of a gene of interest, rather than by their cluster label.

Folder: ~/Desktop/xenviewer_test/Br0942/ — one sample from a Xenium dataset.

- Br0942_test_obj.RDS: processed, annotated SpatialExperiment. Cell-level data only (no transcripts).
  Cell centroids are in full-resolution spatial coordinates. A low-res H&E image is embedded in imgData.
- Br0942.tiff: should be the same low-res H&E as in imgData (~2.4 MB).
- morphology.ome.tif: full-resolution DAPI OME-TIFF from Xenium Ranger (2.8 GB, pyramidal).

Goal: visualize cells overlaid on my choice of the H&E or the DAPI image, colored by cluster, domain, or cell-type annotation (labels transferred from snRNA-seq). Alternatively, allow the user to select a gene to colorize cells by, while also being able to select cells to show/hide based on their cluster labels.

First step: write and run R code that lists what's in the object — colData columns, imgData (scale factors), spatialCoords ranges, reducedDims/altExps — and read the OME-XML metadata of the DAPI file (pixel size, pyramid levels), so we can check that the coordinate systems line up before plotting.
Use R, data.table, and package-supplied functions where they exist for this step. the logcounts matrix requires the `Matrix` package as it is a sparse `dgCMatrix`.

The final visualization tool need not use an R-based tool (e.g., a python package like Vitessece or Napari is fine for the final product). It would be preferable for intermediate steps that prepare the data to be written in R if possible.

Additionally, the final product will probably aim to use a somewhat-downsized H&E image (but not to the degree the test image is downsized).

One other point of reference is that the H&E "scale factor" and Xenium DAPI scale factor are not identical, because they come from different imaging platforms. A lab image analyst previously

determined that a scale factor of 0.25 (µm/pixel? pixel/µm?) and 0.21 to 0.211 resulted in almost perfectly-aligned images. Which scale factor goes with which image modality I don't recall.

Claude · 2026-10-14 02:02 UTC

- ▸ Bash — List data folder and check R packages
- ▸ Bash — Check available TIFF tools and more R packages
- ▸ Bash — Inspect DAPI OME-TIFF directory structure
- ▸ Bash — Test reading TIFF metadata with tiff package
- ▸ Write — Br0942/01_inspect_inputs.R
- ▸ Bash — Run the inspection script
- ▸ Bash — Compare embedded vs standalone H&E images
- ▸ Read — scratch_he_compare.png
- ▸ Bash — Locate offset between embedded and file H&E
- ▸ Bash — Tissue bounding boxes in both H&E versions
- ▸ Bash — Check GDAL view of DAPI file
- ▸ Bash — Check JPEG2000 decoder availability
- ▸ Bash — Replace the IFD reader with an overflow-safe version
- ▸ Read — Br0942/01_inspect_inputs.R
- ▸ Edit — Br0942/01_inspect_inputs.R
- ▸ Read — Br0942/01_inspect_inputs.R
- ▸ Bash — Add cross-check and overlay plots, rerun

Inventory and pyramid parsing now work; checking the overlay plots.

- ▸ Bash — Silence stray prints in script
- ▸ Read — Br0942/01_alignment_check.png
- ▸ Read — Br0942/01_alignment_zoom_dapi.png

The overlay plots exposed two problems. The cell points are too small to see, and the DAPI decode is wrong: most tiles come out black and the rest are only partly filled. Debugging the tile decode.

- ▸ Bash — Inspect a single DAPI JPEG2000 tile
- ▸ Bash — Inspect a single DAPI JPEG2000 tile
- ▸ Bash — Use z max projection for DAPI and rerun
- ▸ Read — Br0942/01_alignment_check.png
- ▸ Read — Br0942/01_alignment_zoom_dapi.png

User · 2026-10-14 02:07 UTC

[Request interrupted by user]

Gah! Stopping you there, I should've given you the maximum intensity projection to start. I will let you know once it's been downloaded and placed in the directory. Please stand by

---

Claude · 2026-10-14 02:07 UTC

I'll wait for the max-intensity projection. When you tell me it's in place I'll point `01_inspect_inputs.R` at it, drop the z-projection code, and rerun.

The coordinate systems line up. The log is in 01_inspect_inputs.log and the overlays are 01_alignment_check.png and 01_alignment_zoom_dapi.png.

Cell centroids are in µm, in the DAPI image's frame.

- At 0.2125 µm/pixel, the full-resolution DAPI image (22913 × 24019 px) is 4869 × 5104 µm.
- The embedded low-res H&E (763 × 800 px) divided by its scale factor (0.15674) gives the same 4868 × 5104.
- To place a cell: DAPI pixel = µm ÷ 0.2125, and low-res H&E pixel = µm × 0.15674.
- At the zoomed-in DAPI level, the centroids sit right on the nuclei.

Your two scale factors:

- **0.2125 µm/pixel is the Xenium DAPI.** It's the pixel size in the file's own metadata, so it's your 0.21–0.211.
- **0.25 µm/pixel is probably the H&E scanner's native resolution** (typical for a 40× slide scan). That fits the evidence: the tissue in `Br0942.tiff` is about 0.83× the size of the tissue in the embedded image, close to 0.2125 ÷ 0.25 = 0.85.

`Br0942.tiff` is not the same image as the H&E embedded in the object. It has the same 763 × 800 canvas, but the tissue is about 17% smaller and shifted. Applying the embedded image's scale factor to it puts cells outside the tissue (middle panel of the overview plot). The embedded image is the one that's already aligned to the cells. For the final, larger H&E we'll need whatever transform produced that embedded image, either an alignment matrix or the ~0.85 scale plus an offset. If the image analyst kept an alignment file, it would answer this.

Other things to know for the viewer:

- The DAPI file is an 11-plane z-stack (3 µm spacing) with 7 pyramid levels, compressed as JPEG 2000. Neither R's `tiff` package nor this machine's GDAL can read that compression directly, so I decoded it tile by tile. Napari and Vitessce will need it converted (e.g. to OME-Zarr) either way.
- Labels available for colouring cells:
  - `BanksyCluster_Label` (16 clusters, including `DROPPED_CLUSTER`)
  - `Xfered_snRNA_Label` (24 labels transferred from snRNA-seq)
  - `Xfered_SRT_Label` (9 labels)
- Domains for outlining: 6 two-level LC/Other columns, `SmoothedLC{8,12,16,20}_SRTxfer` and `SmoothedLC{8,12}_snRNAxfer`. Each covers about 1.9–2.2k cells.
- Gene expression: `logcounts` covers 366 genes × 21,923 cells, stored as a sparse matrix that is 23% non-zero.

User · 2026-10-14 03:07 UTC

Ok, now the maximum intensity projection is in there, `morphology_focus_0000.ome.tif` , but it's going to have the same stupid JPEG2000 problem, I suspect. If using FIJI's CLI is helpful, I can put my modified copy of Silicon-native FIJI into your working directory (which has been tweaked

to get Bio-Formats to read JPEG2000 OME-TIFFs, which is currently broken in FIJI as shipped.) Let me know if that would be helpful for wrangling the max intensity projection.

Claude · 2026-10-14 03:07 UTC

▸ Bash — Inspect the new focus/MIP OME-TIFF structure
▸ Bash — Print pyramid of the focus image

Same 22913 × 24019 grid and pyramid as the z-stack, now a single plane, still JPEG 2000. Re-pointing the script at it.

▸ Bash — Point script at focus image and rerun
▸ Read — Br0942/01_alignment_zoom_dapi.png

Thanks, but I don't need FIJI for this. The R script already reads `morphology_focus_0000.ome.tif` as it is: it pulls out each 1024 × 1024 JPEG 2000 tile and decodes it with GDAL (through the `sf` package). I've pointed 01_inspect_inputs.R at the focus image and rerun it.

- **Same grid as the z-stack.** It's 22913 × 24019 px at 0.2125 µm/px, with the same 7-level pyramid. It's a single plane, so the max-projection step now does nothing.
- **Alignment still holds.** In the new zoom plot (01_alignment_zoom_dapi.png) the nuclei are sharper than with my projection, and the centroids still land on them. So cell µm ÷ 0.2125 = DAPI pixel, with no offset.

Converting it for the viewer. Napari and Vitessce will want an OME-Zarr or OME-TIFF pyramid that isn't JPEG 2000. There are two ways to do that without FIJI:

- **R + GDAL:** decode the 552 full-resolution tiles the same way I do now, then write them out. R has no good tool for writing OME-Zarr or tiled OME-TIFF, though, so this would be awkward.
- **Python `tifffile` + `imagecodecs` (my recommendation):** `imagecodecs` reads JPEG 2000 natively. This is a short, single-purpose script in a small virtual environment, and it can also build the downsized H&E pyramid. The cell, label, domain-outline and expression tables would still be prepared in R.

Your FIJI build would be a useful fallback if a file turns up that this approach can't read, so keep it handy. I don't need it for now.

One open question before the viewer build: where does the "somewhat-downsized H&E" come from? It has to land in the same frame as the cells. Either it's derived the same way as the embedded image (and `Br0942.tiff` doesn't match that), or we need the H&E → Xenium alignment transform from your image analyst.

User · 2026-10-14 03:13 UTC

It's actually quite curious that the `Br0942.tiff` file is maligned. You can ignore it or delete it. If the image in the spatialexperiment object lines up with the maximum intensity projection image (which

it sounded like was the case earlier on), I probably tried to grab it out of the SPE object and saved it wrong as the standalone TIFF.

(I do have the code to downsize the full-res H&E images and put them on the imgData slot, like for the sample we're currently working with. So if the image in the SPE lines up properly with the MIP but I decide I want a higher resolution H&E image, it will be trivial to make a new SPE object with that higher res image without the alignment issues)

Claude · 2026-10-14 03:13 UTC

I'll ignore `Br0942.tiff` and leave the file in place. Before relying on the embedded H&E, I'll check it directly against the DAPI focus image, since so far I've only compared each image to the cells.

▸ Bash — Directly compare embedded H&E with DAPI focus image
▸ Bash — Fix smoothing and rerun H&E vs DAPI check
▸ Read — /private/tmp/claude-502/-Users-bmulvey-Desktop-xenviewer-test/25e05e33-0e47-4c2c-9ebf-e4e74990464a/scratchpad/he_dapi.png

The embedded H&E lines up with the DAPI focus image: the best shift is 0,0 (one low-res H&E pixel is 6.4 µm). I'll fold this check into the script and drop `Br0942.tiff` from it.

▸ Bash — Fold H&E-vs-DAPI check into script, drop Br0942.tiff

The H&E stored in the SPE object lines up with the DAPI focus image, so your plan works. Swapping in a higher-resolution H&E with your own code shouldn't cause alignment problems.

How I checked: I mapped both images onto the same micrometre grid and compared where each shows tissue. The overlap is best with no shift at all (overlap score 0.88 at zero shift). Shifting by 1 pixel in any direction makes the overlap slightly worse. In the overlay, the DAPI nuclei fill the H&E tissue, and the hole and torn edges match.

That only rules out offsets of about one low-res pixel, which is 6.4 µm. When you build the higher-resolution version, rerun the script to confirm alignment at that finer scale.

I've added this check to 01_inspect_inputs.R as section 10. The script no longer references `Br0942.tiff`; I left the file itself in place for you to delete. It now produces:

- 01_alignment_check.png: cells over the H&E and over the DAPI image
- 01_alignment_zoom_dapi.png: close-up showing centroids on nuclei
- 01_alignment_he_vs_dapi.png: H&E with DAPI nuclei overlaid in green
- 01_inspect_inputs.log: the full inventory

Coordinates for the viewer: cells are in micrometres. A DAPI pixel is µm ÷ 0.2125. An H&E pixel is µm × the scale factor stored with the image in the SPE. No offsets are needed.

Next step, in R:

- Export a cell table with centroids and all the label columns.
- Store gene expression from `logcounts` in a sparse format that loads quickly.

- Turn each domain column into outline polygons using the `sf` and `concaveman` packages, which are both installed.

After that comes the image conversion and the viewer. Before I build it, do you have a preference between Napari and Vitessce?

- **Napari** is a desktop app on your machine.
- **Vitessce** runs in a web browser, so it's easier to share.

User · 2026-10-14 03:16 UTC

Vitessce. The end goal will be to have a Vitessce instance we can host from our institute's server for all 30-whatever samples in this dataset, so browser-ready is a must. Go ahead with the R data prep!

Claude · 2026-10-14 03:17 UTC

▸ Bash — Check R packages and Python tooling
▸ Bash — List anndataR exports and conda envs
▸ Bash — Plot domain label maps and cell spacing
▸ Read — /private/tmp/claude-502/-Users-bmulvey-Desktop-xenviewer-test/25e05e33-0e47-4c2c-9ebf-e4e74990464a/scratchpad/dom.png
▸ Bash — Check anndataR conversion API
▸ Write — 02_prep_viewer_data.R
▸ Bash — Run viewer data prep on Br0942

Two fixes needed: the Glasbey palette tops out at 32 colours, and one geometry type conversion failed.

▸ Bash — Fix palette and geometry cast, rerun
▸ Read — viewer_data/Br0942/qc_domains.png

The outlines are in the right place, but each one is full of small interior squiggles that should have been filled. Debugging the hole-filling step.

▸ Bash — Count rings and hole areas in a domain polygon
▸ Bash — Inspect geometry types in domain GeoJSON
▸ Bash — Inspect rings of the dissolved Voronoi domain
▸ Read — 02_prep_viewer_data.R
▸ Bash — Rework tissue mask and outline cleanup, rerun

Runtime is down from 95 s to 8 s. Checking the outlines:

- ▸ Read — viewer_data/Br0942/qc_domains.png
- ▸ Bash — Validate h5ad with Python anndata
- ▸ Bash — Show sample metadata and GeoJSON head

The R data prep is done. 02_prep_viewer_data.R ran on Br0942 in about 8 seconds, and the `.h5ad` it wrote loads correctly in Python's `anndata`.

To run it on another sample:

```
Rscript 02_prep_viewer_data.R Br0942/Br0942_test_obj.RDS Br0942/
morphology_focus_0000.ome.tif viewer_data
```

It writes these files to `viewer_data/Br0942/`:

File	Contents
`cells.h5ad` (17 MB)	Gene expression from `logcounts`, raw `counts`, and cell centroids in µm. It also holds the 3 label columns and 6 domain columns as categories, which Vitessce turns into show/hide cell sets, plus each label's colours.
`domains.geojson` (74 KB)	One outline per domain column, in the same µm coordinates as the cells. Each piece records its column, level, how many cells it contains, its area and its colour.
`he_image.tif`	The H&E from the SPE object, plus its pixels-per-µm in `sample_meta.json`
`sample_meta.json`	Scale factors, image sizes, which columns are labels vs domains, and the outline settings
`qc_domains.png`	The outlines drawn over the H&E, for checking by eye

There's also one shared file, `viewer_data/label_palettes.json`, used across all samples. Each label keeps the same colour in every sample, and labels that first appear in a later sample get new colours added. The current colours are bright pure red, green and magenta, which are harsh on H&E. You can edit them by hand in that file and it will keep your choices.

How the domain outlines work:

1. Each cell gets the patch of tissue closer to it than to any other cell (a Voronoi tessellation).
2. I merge the patches of the LC-labelled cells. The boundary therefore sits halfway between LC cells and their Other neighbours.
3. The merged region is clipped to the tissue, so it follows real tears such as the big hole.
4. Pieces with fewer than 10 LC cells are dropped, and holes containing fewer than 10 cells are filled.
5. The edges are smoothed.

All of these settings are at the top of the script. For example, `SmoothedLC8_SRTxfer` keeps two small satellite patches and `SmoothedLC16_SRTxfer` keeps one, and every cell labelled LC (or all but a few) ends up inside an outline. The domain columns also stay on each cell, so you can still filter cells by domain if you want.

Next is the Python step, which only converts formats:

- Convert `cells.h5ad` to zarr, with expression laid out so that colouring by a gene loads quickly.
- Convert the DAPI image and the H&E into multi-resolution OME-Zarr images the browser can stream.
- Convert the domain outlines into an outline-only layer.
- Write a Vitessce config with a toggle between H&E and DAPI, colour-by-label or colour-by-gene, and cell-set show/hide.

I'll set up a small `uv` Python environment for it and check everything locally in the browser before we think about the server.

User · 2026-10-14 03:47 UTC

Regarding cluster label palettes, I have added a 9-color palette that matches the one we used in the paper for the SRT labels (and transferred SRT labels) to the top-level directory, `finalpal_somuchfornorainbows.RDS`. Each color is named for one of the transferred SRT labels, or should be close to it (the names in the palette may have markdown characters for ggplot purposes but should be edited to match the values of the SRT labels found in the SPE).

A 15-color palette for the 15 Banksy clusters is also now there, in the same configuration (named vector of hex colors): `banksypal.RDS`

The color of the domain outline (regardless of which domain is being displayed) should be the same blue as the SRT palette's `LC` entry.

Claude · 2026-10-14 03:47 UTC

▸ Bash — Inspect the supplied palettes vs label values
▸ Write — 00_build_palettes.R
▸ Bash — Apply paper palettes, rerun prep

All the SPE labels matched a palette entry, and no warnings were printed. Checking that the colours stored in the `.h5ad` line up with the label order, and looking at the QC plot:

▸ Bash — Verify colours in h5ad and GeoJSON
▸ Read — viewer_data/Br0942/qc_domains.png

Your two paper palettes are now applied throughout. All domain outlines are drawn in the SRT palette's LC blue (`#0609C6`).

What changed:

- **New 00_build_palettes.R.** It reads your two RDS palettes and renames the display names to the SPE label values, using an explicit lookup table. It stops with an error if a palette entry has no mapping. It writes the result to `viewer_data/label_palettes.json`. Run it once for the project; `02_prep_viewer_data.R` keeps those colours and only adds colours for labels missing from them.
- **Name matching.** The 9 SRT names matched directly; for example `*ACHE*, *SERT*, LC, Oligo Mixed` became `ACHE_SERT_LC_WM` and `Oligo and Other Types Mixed` became `Oligo_Complex`. For Banksy, `NK/T Cells` became `NK_and_T_cells`, `GABA *5HTR2A*` became `Inhib_5HTR2A`, `5HTergic` became `Serotonergic`, and so on. Every Br0942 label found a colour.
- **`DROPPED_CLUSTER`** isn't in the 15-colour Banksy palette, so I gave it dark grey (`#4D4D4D`). That keeps it distinct from `Oligodendrocytes` (`#BCBCBC`). Change it in `00_build_palettes.R` if you want something else.
- **Domain columns** use the LC blue for the domain itself and light grey for `Other`. That applies to the outlines and also when a domain column is used to show or hide cells.
- **New-label warning.** If a later sample has a label the paper palettes don't cover (a renamed cluster, say), it gets an automatic colour and the script prints a warning, rather than failing silently.
- **`Xfered_snRNA_Label`** has no paper palette, so its 24 labels still get automatic colours. If you have a palette for it too, it's a two-line addition.

Checks: the colours stored in `cells.h5ad` match the palette for every label, and all 9 outline pieces in `domains.geojson` carry the blue.

In the QC plot, Blood cells (red) sit in the vessels and Ependyma (purple) sits at the upper-left. So the cell positions and their colours line up with the tissue.

Next is the Python packaging step: zarr for the cells, OME-Zarr image pyramids for DAPI and H&E, the outline layer, and the Vitessce config.

User · 2026-10-14 04:17 UTC

Just making sure, there isn't an "Other" polygon being drawn, right? The gray assigned there is just because some color needed to be assigned? (There should only be a polygon that encompasses the "LC" cells for each domain specification. The "Other" is implicitly everything else so "Other" polygons are unnecessary, if they're currently being produced for some reason.)

Claude · 2026-10-14 04:17 UTC

▸ Bash — List domain/level pairs in the GeoJSON

Right, no "Other" polygon is drawn. All 9 shapes in `domains.geojson` are LC outlines: one per domain column, plus the small satellite patches for LC8_SRTxfer (3 pieces total) and LC16_SRTxfer (2 pieces).

The script skips "Other" when it builds outlines (`domain_background <- "Other"` at the top of 02_prep_viewer_data.R).

The grey is only there because each cell still carries its LC/Other label in `cells.h5ad`. In Vitessce, those labels let you show or hide cells by domain, and every category in that list needs a colour. "Other" is never drawn as a shape.

User · 2026-10-14 04:19 UTC

Cool cool. Then let ‘er rip with the Python packaging and Vit config’ing!

Claude · 2026-10-14 04:19 UTC

I’ll package everything into browser-ready static files and test locally in the browser pane. First, setting up a pinned Python environment and checking what the current `vitessce` Python package provides.

- ▸ Bash — Create uv project and install packaging deps

Environment is ready (vitessce 3.9.7, zarr 3.4, imagecodecs). Next I’ll check which file types and helper utilities this version supports.

- ▸ Bash — Inspect vitessce package wrappers and constants
- ▸ Bash — Inspect data_utils functions and widget JS source

Vitessce JS here is v4.0.8. Reading how the widget loads it (to reuse for a standalone page) and the wrapper signatures:

- ▸ Bash — Read widget JS loading code
- ▸ Bash — Read OME-Zarr and segmentation wrapper code
- ▸ Bash — Read AnnData wrapper and OME-Zarr writers
- ▸ Bash — List config-building API
- ▸ Bash — Read coordination-dict docs and coordination types

The Python enum lags the JS app, so I’ll pull the coordination-type list for Vitessce JS 4.0.8 straight from npm (read-only reference):

- ▸ Bash — Fetch Vitessce constants package for reference
- ▸ Bash — List spatial coordination types in JS constants
- ▸ Bash — Check spatialBeta view and file types
- ▸ Bash — Read spatialBeta coordination list
- ▸ Bash — Read spatialBeta coordination types
- ▸ Bash — Remaining spatialBeta coordination; look for example configs package

Next I’ll pull reference spatialBeta example configs from the Vitessce GitHub repo (v4.0.8 tag) to match the config format exactly:

- ▸ Bash — List spatial-beta example configs on GitHub
- ▸ Bash — Download and read visium-spots example config
- ▸ Bash — Read Xenium spatialdata example config
- ▸ Bash — Scan KPMP bitmask segmentation example

- Bash — Read KPMP config structure
- Bash — Check vitessce npm entry points
- Bash — Inspect bare imports in vitessce bundle
- Bash — Check vitessce API and tifffile pyramid levels
- Write — 03_package_vitessce.py
- Bash — Run packaging for Br0942

Packaging took 10 s. I'll pin `anndata<0.14`, because 0.14 drops the Zarr v2 writer. Then I'll check the outputs and the generated config.

- Bash — Inspect packaged outputs and config
- Write — site/index.html
- Bash — Create launch config for local static server
- Skill — anthropic-skills:built-in-browser
- browser:preview_start — xenviewer-site
- browser:computer — screenshot
- browser:computer — wait
- browser:read_console_messages
- browser:computer — screenshot
- browser:read_network_requests
- browser:resize_window
- browser:navigate — http://localhost:8765/
- browser:computer — screenshot
- browser:computer — wait
- browser:computer — zoom
- browser:computer — screenshot
- browser:computer — zoom

Each domain definition shows up as its own toggleable entry in the layer panel, which is what we wanted. Next I'll read the whole panel:

- browser:get_page_text
- browser:javascript_tool — javascript_exec
- browser:computer — scroll
- browser:computer — screenshot
- browser:computer — scroll
- browser:computer — screenshot

At every zoom level the outline renders as a band roughly 100 µm wide, not a thin line. To find out why, I'll check which resolution of the domain mask the browser is actually loading.

- browser:javascript_tool — javascript_exec
- browser:javascript_tool — javascript_exec
- browser:computer — zoom

I'll test how the stroke width behaves by setting it to 1 and reloading:

- ▸ Bash — Set domain stroke width to 1 for testing
- ▸ browser:navigate — http://localhost:8765/
- ▸ browser:computer — wait
- ▸ browser:computer — screenshot
- ▸ browser:computer — scroll
- ▸ browser:computer — screenshot

Stroke width 1 gives a clean, visible outline, so 3 was simply too thick. Next I'll check how it looks zoomed in:

- ▸ browser:computer — scroll
- ▸ browser:computer — screenshot
- ▸ Bash — Inspect Vitessce bitmask stroke-width shader
- ▸ Bash — Read bitmask outline shader logic
- ▸ Bash — Switch domains to drawn outline rings and repackage
- ▸ browser:navigate — http://localhost:8765/
- ▸ browser:computer — wait
- ▸ browser:computer — screenshot

Now the outline is a thin, continuous line at the full-section view. Next I'll zoom in:

- ▸ browser:computer — scroll
- ▸ browser:computer — screenshot
- ▸ browser:browser_batch — computer; computer; computer; computer
- ▸ browser:find — Search
- ▸ browser:browser_batch — Focuses the gene search box; Types TH into the gene search box; computer; find
- ▸ browser:computer — screenshot
- ▸ browser:browser_batch — Selects gene TH in the gene list; computer; computer

Colouring by gene works: TH-expressing cells (green and yellow) sit inside the LC outline. Next I'll test hiding clusters while the gene colouring stays on.

- ▸ browser:browser_batch — Expands the Banksy cluster set list; computer; computer
- ▸ browser:find — checkbox
- ▸ browser:find — Oligodendrocytes
- ▸ browser:read_page
- ▸ browser:javascript_tool — javascript_exec

Each cell set has a checkbox, all checked by default. I'll uncheck Oligodendrocytes (the largest cluster) while TH colouring is active and see whether those cells disappear:

▸ browser:javascript_tool — javascript_exec
▸ browser:javascript_tool — javascript_exec
▸ browser:browser_batch — Unchecks the Oligodendrocytes cell set; computer; computer
▸ browser:javascript_tool — javascript_exec
▸ browser:browser_batch — Unchecks the Oligodendrocytes cell set checkbox; computer; javascript_tool; computer
▸ Bash — Fetch spatial-beta and obs-sets packages for reference
▸ Bash — Search spatial-beta for set-selection handling

I'm checking whether the spot layer honours a cell filter (`obsFilter`). If it does, show/hide by cluster can be implemented independently of the colouring.

▸ Bash — Check obsFilter support and getCellColors logic
▸ Bash — Read spot layer rendering code
▸ Bash — Trace spot color computation and default color
▸ Bash — Read expression extension shader for alpha handling
▸ Bash — Read ScaledExpressionExtension shader module
▸ Bash — Read spot colour getter and set-colour mapping
▸ Bash — Read spot set-colour update logic
▸ Bash — Download vitessce dist and locate spot colour getter
▸ Bash — Find spot layer code in vitessce dist chunks
▸ Bash — Read the bundled spot colour getter
▸ Bash — Check getter usage and bare imports in dist
▸ Write — 04_vendor_vitessce.py
▸ Bash — Vendor patched Vitessce and point the page at it

Updating the comment in the page to match:

▸ Edit — site/index.html
▸ browser:browser_batch — navigate; computer; read_console_messages; computer
▸ browser:browser_batch — computer; computer; Expands the Banksy cluster set list; find
▸ browser:browser_batch — Focuses the gene search box; Types TH into the gene search box; computer; Selects gene TH to colour cells by expression; computer; javascript_tool
▸ browser:computer — screenshot

ABCC9 got selected instead of TH because of a stale element reference. I'll pick TH properly:

▸ browser:find — Search textbox
▸ browser:browser_batch — Focuses the gene search box; Types TH into the gene search box; computer; computer

Now I'll click TH, then untick the parent "Banksy cluster" checkbox, which unticks all of them:

▸ browser:browser_batch — Selects gene TH to colour cells by expression; computer; javascript_tool
▸ browser:computer — zoom

The parent row has no checkbox, so I'll untick the leaves individually. First, the positions of the visible ones:

▸ browser:javascript_tool — javascript_exec
▸ browser:browser_batch — Unticks Astro_FGFR3; Unticks Astro_RYR3; Unticks Astro_SERPINA3; Unticks Blood; computer; javascript_tool; computer

Unticking works, but it also switches the colouring back to clusters, which is Vitessce's built-in behaviour. To test the combination, I'll re-select TH now that 4 clusters are unticked:

▸ browser:browser_batch — Selects gene TH to colour cells by expression; computer; computer

Gene colouring is back. To make the hiding unambiguous, I'll untick everything except LCNE:

▸ browser:browser_batch — Unticks DROPPED_CLUSTER; Unticks Endothelium; Unticks Ependyma; Unticks Inhib_5HTR2A; computer; computer; javascript_tool
▸ browser:browser_batch — Unticks Mixed Neuropept.; Unticks NK_and_T_cells; Unticks OPC; Unticks Oligodendrocytes; Unticks Serotonergic; Unticks Vasc. SMC; computer; computer; javascript_tool
▸ browser:browser_batch — Unticks Microglia, leaving only LCNE; computer; computer; Re-selects gene TH to colour the remaining cells by expression; computer; computer
▸ browser:javascript_tool — javascript_exec
▸ browser:browser_batch — computer; computer; computer
▸ browser:browser_batch — Turns on the DAPI image layer; Turns off the H&E image layer; computer; computer; computer

The DAPI layer works. Zooming in further to check that cell markers sit on nuclei:

▸ browser:browser_batch — computer; computer; computer

Only LCNE cells are showing here, and their large, dim neuronal nuclei are hard to judge against DAPI. For a fair alignment check I'll reload (all clusters shown), switch to DAPI and zoom right in:

▸ browser:browser_batch — navigate; computer; computer; computer; computer
▸ browser:browser_batch — computer; computer; computer; Turns on the DAPI image layer; Turns off the H&E image layer; computer; computer; computer; computer; computer; computer

The LCNE cells (blue) sit inside the domain outline. To check registration against nuclei, I'll hide the cell markers and compare the same field:

▸ browser:browser_batch — computer; computer; Hides the cell marker layer; computer; computer

Cell positions match the nuclei pattern, consistent with the R check. The 5 µm marker radius is a bit larger than most nuclei, so I'll drop it to 4 µm and regenerate the config only:

▸ Bash — Set spot radius to 4 µm and regenerate config

Last check: toggling a second domain definition on from the layer panel.

▸ browser:browser_batch — navigate; computer; computer; Turns on the SmoothedLC8_snRNAxfer domain outline; computer; computer; computer

Two domain definitions can be shown at once, and the legend lists both. Everything I set out to test works. I'll reset the browser viewport and write a short README with the run order.

▸ browser:resize_window
▸ Write — README.md
▸ Bash — List final project layout

The Vitessce viewer for Br0942 is built and running locally at http://localhost:8765, in the browser pane. I clicked through every feature there:

- **Background image:** H&E or DAPI, switched from the layer panel. The cells, outlines and both images line up; at high zoom the cell markers match the DAPI nuclei.
- **Domain outlines:** each of the 6 domain definitions is its own on/off entry in the layer panel, drawn as a thin LC-blue line. The first one, SmoothedLC8_SRTxfer, is on when the page opens. The outline encloses the dark neuromelanin-containing LC neurons visible in the H&E, a nice independent check.
- **Colour by cluster:** Banksy clusters, transferred SRT labels, transferred snRNA-seq labels, or domain membership, using your paper palettes.
- **Colour by gene:** pick any of the 366 genes in the Gene List to colour cells by `logcounts`. TH lights up inside the LC outline.
- **Show/hide by cluster while colouring by gene:** I tested this with only LCNE ticked and TH selected: just the 923 LCNE cells were drawn, coloured by TH. Ticking or unticking a cluster flips the colouring back to clusters, which is Vitessce's normal behaviour; click the gene again to return to gene colouring.

Two workarounds you should know about:

1. **I patched Vitessce for show/hide.** Out of the box, Vitessce 4.0.8 can't hide clusters: unselected cells are drawn grey, and gene colouring ignores the cluster selection entirely. So `04_vendor_vitessce.py` downloads Vitessce 4.0.8 into the site and changes one small function

so unselected cells are transparent. The script checks the download against npm's checksum, and it stops with an error if a future Vitessce version no longer contains the code it patches. When upgrading Vitessce, the patch has to be re-checked.

2. **The domain outlines are drawn into the image data.** Vitessce's own outline mode draws a line about 40 µm wide that grows when you zoom in and disappears when you zoom out. Instead, I draw a 3 µm line into the domain masks myself, with the zoomed-out versions built so the line never breaks up. It stays thin at every zoom.

Pipeline per sample:

```
Rscript 02_prep_viewer_data.R Br0942/Br0942_test_obj.RDS Br0942/
morphology_focus_0000.ome.tif viewer_data
uv run python 03_package_vitessce.py viewer_data/Br0942
```

Run `Rscript 00_build_palettes.R` and `uv run python 04_vendor_vitessce.py` once for the whole project. The Python step takes about 10 seconds per sample. Run order and details are in README.md.

For deploying to your institute's server:

- **What to copy:** the whole `site/` folder, as plain static files with no server-side code. It's about 485 MB per sample, almost all of it the DAPI image (stored lossless), so 30 samples comes to roughly 15 GB.
- **Sample list:** each new sample is added to the page's sample picker automatically.
- **Internet dependency:** React still loads from a public CDN (esm.sh, pinned version). If the server mustn't depend on outside CDNs, React can be bundled in the same way as Vitessce.
- **Outline colour:** all domain outlines are the same blue, as you asked. When two are switched on at once, you can recolour either one from the colour swatch next to it in the layer panel.

User · 2026-10-14 17:23 UTC

Let's do a little fine tuning with the interface.

1. Let's group the 6 domains into a dropdown menu in the righthand sidebar, where the user can select one of the six LC domain outlines to visualize (or unselect them altogether) in a menu named "LC Region Boundaries" to visualize, if the user so desires.
2. Add a slider to modify cell sizes.
3. Include a color legend for gene expression.
4. Extend the gene expression colorization to allow for multiple genes to be selected, in which case the spots should be colorized by the mean of those genes' Z-scored logcounts.
    1. When multiple genes are selected, add a small options toggle that would allow for Z-scaling within the current sample or across all samples (but we'll start on adding a second sample to this viewer later).
5. Change the sample name that appears in the dropdown menu to be BrNNNN(L)*[APOEgenotype]* [Sex]_[Age rounded to nearest integer]. Additionally have a centered title over the viewer pane that lists the current brain being displayed and shows the same information formally, e.g.. "Br9999 (APOE4 homozygous, Female, 64 y/o)"

Claude · 2026-10-14 17:25 UTC

I'll start with the current state of `03_package_vitessce.py`, since it changed on disk, and with what Vitessce 4.0.8 already provides for multi-gene aggregation and its legend.

- Read — 03_package_vitessce.py
- Bash — Read Vitessce expression aggregation hooks

The reference packages in my scratchpad are gone, so I'll re-fetch the ones I need: spatial view, legend, gene list, and the plugin API.

- Bash — Re-fetch Vitessce reference packages; read aggregation hooks
- Bash — Read Vitessce feature aggregation implementation

Good: `mean` averages the raw values, so pointing it at a z-scored matrix gives exactly the mean of z-scores. Next I'll check what the built-in legend shows:

- Bash — Read the spatial legend component
- Bash — Read Vitessce plugin class definitions
- Bash — Read useCoordination hook and plugin registration
- Bash — Check Vitessce plugin props and scope resolution

I'll check how the Python config builder attaches meta-coordination scopes. The domain layer needs to be linked only to the spatial view (so it leaves the layer panel), while my dropdown controls it.

- Bash — Read python meta-coordination helpers
- Bash — Read use_meta_coordination body

The Python builder appends meta-coordination scopes, so the domain layer can link to the spatial view alone. One more check: the gene list's multi-select prop name in 4.0.8.

- Bash — Check featureList multi-select and view prop passing
- Bash — Locate metadata block in R prep script
- Bash — Export donor metadata from R and rerun prep

User · 2026-10-14 17:28 UTC

(sent while Claude was working)

Also a note that, where feasible, you are welcome to allocate up to 8 CPU cores from this machine to R or python steps and/or use apple silicon-compatible GPU acceleration. The ability to parallelize will make building the full multisample app much simpler

---

Claude · 2026-10-14 17:28 UTC

▸ Bash — Add z-score layers, control-panel wiring and labels to packager

Now `05_global_zscores.py`, which computes pooled per-gene statistics across all packaged samples and rewrites each sample's across-samples z-score layer, using up to 8 worker processes:

▸ Write — 05_global_zscores.py
▸ Bash — Repackage cells and compute global z-scores; validate

For a single sample, the "all samples" z-scores should equal the within-sample ones, but `allclose` says they differ. Checking by how much:

▸ Bash — Measure z-score difference between layers

A relative difference of about 0.03%, from float32 vs float64 accumulation. It's harmless, but I'll compute both in float64 so they agree exactly:

▸ Bash — Compute z-scores in float64 and re-verify
▸ Bash — Pass cells store URL to control panel props
▸ Read — site/index.html

I'll put the control panel in its own module (`site/xenviewer-controls.js`) to keep the page readable:

▸ Write — site/xenviewer-controls.js

Next, the page: labels in the picker, the title row, plugin registration, and URL resolution for the plugin's props.

▸ Write — site/index.html
▸ Bash — Regenerate config and inspect control view wiring

The scope name `A` for `spatialTargetC` might clash with the H&E channels' auto-generated scopes. Checking:

▸ Bash — Inspect coordination scope names for clashes

No clash: the image channels got scopes B–E, and scope A belongs to the domain channel. On to testing in the browser:

▸ browser:preview_start — xenviewer-site
▸ browser:browser_batch — resize_window; navigate; computer; read_console_messages; computer

The layout is broken: the right-hand panels are stacked on top of each other and overlap the spatial view. I'll inspect the grid item positions and the 404s:

▸ browser:javascript_tool — javascript_exec

The grid positions are actually correct; the screenshot caught a layout animation mid-way. The 404s are the usual harmless Zarr probes. Fresh screenshot:

▸ browser:computer — screenshot
▸ browser:find — LC Region Boundaries combobox
▸ browser:find — combobox
▸ browser:browser_batch — Selects the SmoothedLC20_SRTxfer LC boundary; computer; computer; Turns off LC boundary outlines; computer; computer; Selects the SmoothedLC8_SRTxfer LC boundary; computer; computer

The dropdown works: SmoothedLC20_SRTxfer draws its single region, None hides the outline (and its legend entry), and LC8 brings back the version with satellite patches. Next, the cell-size slider:

▸ browser:find — slider
▸ browser:find — Cell size

The slider has no accessible name, so I'll add `aria-label`s to all the panel controls, then reload and drag it:

▸ Bash — Add aria-labels to control panel inputs
▸ browser:browser_batch — navigate; computer; find
▸ browser:browser_batch — computer; computer; computer; computer; computer; javascript_tool
▸ browser:browser_batch — Sets the cell size slider to about 11 μm; computer; computer; computer

Cell markers grow to 11 μm as expected. Next, multi-gene selection. I'll reload, search TH, and check how the multi-select gene list renders:

▸ browser:browser_batch — navigate; computer; computer

Five seconds after load, the panels are still overlapping, so this isn't just a quick animation. I'll check the grid's actual state:

▸ browser:javascript_tool — javascript_exec

The DOM positions are correct, so this is likely a rendering artifact in my capture rather than the page. I'll check whether the browser pane is hidden, which would pause painting:

‣ browser:tabs_context
‣ browser:computer — screenshot
‣ Bash — Disable grid slide-in transition
‣ browser:browser_batch — navigate; computer; find
‣ browser:computer — screenshot

With the transition off, the layout is correct on the first capture. Now I'll search for TH and look at the multi-select gene list:

‣ browser:browser_batch — Focuses the gene search box; Types TH into the gene search box; computer; computer; javascript_tool
‣ browser:browser_batch — Selects gene TH; computer; computer

The new legend reads "TH · logcounts, 0.00 – 6.06", matching Vitessce's own inset legend (0.0 – 6.1). Now I'll add a second gene, DBH:

‣ browser:browser_batch — Selects the text in the gene search box; Types DBH into the gene search box; computer; computer
‣ browser:browser_batch — Adds gene DBH to the selection; computer; computer
‣ Bash — Independently compute legend ranges in Python
‣ browser:computer — zoom

My legend matches Python exactly (–0.340 to 6.274; TH alone 0 to 6.056). The inset legend is too small to read here, so I'll pull its text:

‣ browser:javascript_tool — javascript_exec
‣ browser:javascript_tool — javascript_exec
‣ Bash — Hide inset legend in gene mode via control panel

Re-testing the whole flow:

1. Select TH and DBH, and check that the inset legend disappears.
2. Flip the z-score toggle.
3. Switch back to cluster colouring, and check that the inset legend returns.

‣ browser:browser_batch — navigate; computer; computer; Focuses the gene search box; Types TH into the gene search box; computer; Selects gene TH; computer; Selects the text in the gene search box; Types DBH into the gene sear

The misleading inset legend is gone, and the domain legend entry stays. Next I'll flip the toggle to "all samples" and confirm the viewer loads the across-samples matrix:

‣ browser:browser_batch — Switches z-scoring to all samples; computer; computer; javascript_tool
‣ browser:browser_batch — computer; computer; Expands the Banksy cluster set list; com-

puter; find
‣ browser:browser_batch — Colours only the LCNE cluster; computer; computer
‣ Bash — Add return-to-genes and clear-genes buttons

The combined test: select TH and DBH, keep only LCNE via the set button, then click “Colour by TH, DBH”. Only LCNE cells should show, coloured by mean z-score.

‣ browser:browser_batch — navigate; computer; computer; Focuses the gene search box; Types TH into the gene search box; computer; Selects gene TH; computer; Selects the text in the gene search box; Types DBH into the gene sear
‣ browser:browser_batch — Colours only the LCNE cluster; computer; computer; find
‣ browser:browser_batch — Switches cell colouring back to the selected genes; computer; computer

That works: only LCNE cells are drawn, coloured by the TH+DBH mean z-score, with the legend showing and the inset legend hidden. Last check, “Clear genes”:

‣ browser:browser_batch — find
‣ browser:browser_batch — Clears the gene selection; computer; computer; read_console_messages
‣ browser:resize_window
‣ Bash — Update README for new controls and z-score step

All five changes are in and working at http://localhost:8765. I clicked through each one in the browser pane.

1. LC Region Boundaries. A new “Display options” panel at the top of the right sidebar has a dropdown listing the six domain definitions plus “None”. Choosing one draws only that outline; “None” hides it. The outlines are no longer in the layer panel.

2. Cell size. A slider in the same panel sets the cell marker radius from 1 to 15 μm (default 4).

3. Gene expression legend. The panel shows a colour bar with the real value range, computed from the same numbers that colour the cells. I checked it against Python: TH alone gives 0.00–6.06 logcounts, and TH+DBH gives −0.34 to 6.27 mean z-score.

- Vitessce did already have a legend, but it’s a small inset in the corner. For multiple genes it’s also wrong: it reads “0 – 238”, because it averages each gene’s internal display scale. The panel now hides that inset while colouring by gene, and keeps it for cluster colouring, where it’s useful.

4. Multiple genes. The gene list is now multi-select: click genes to add or remove them. With two or more genes, cells are coloured by the per-cell mean of their z-scored logcounts.

- **Z-score toggle:** “this sample” or “all samples (n = N)”. The across-samples z-scores come from a new step, `05_global_zscores.py`. It pools each gene’s mean and SD over every cell in every packaged sample, using 8 worker processes. With one sample the two options are identical; re-run it whenever samples are added.

- **Two new buttons:** ticking cell sets switches the colouring back to clusters, which is Vitessce's normal behaviour. Getting back to the genes used to mean re-clicking them in the list, which toggles them off. So the panel now has "Colour by TH, DBH" to return to gene colouring, and "Clear genes" to drop the selection.
- I tested the combination: show only LCNE plus TH+DBH colouring draws just the LCNE cells, coloured by mean z-score.

5. Sample naming. The sample picker shows `Br0942_E4hom_M_62`, and the title centred over the viewer reads "Br0942 (APOE4 homozygous, Male, 62 y/o)". The R prep step now exports brnum, APOE, Sex and Age into each sample's metadata. The APOE wording is generated from the code, so `E3E4` would become "APOE3/4 heterozygous".

Question on the label format: I read the "(L)" in `BrNNNN(L)` as an optional letter that's part of the brain number, and used `brnum` exactly as stored. If it means something else, such as hemisphere, tell me which column it comes from and I'll add it.

On the 8 cores:

- `05_global_zscores.py` already runs on 8 worker processes.
- Per sample, the R prep and Python packaging take about 8 s and 10 s. Each sample is independent, so when we add the second sample I'll write a batch driver that runs 8 samples at a time.
- A GPU wouldn't help: the slow parts are image decoding and compression, not matrix maths.

The page code is in site/xenviewer-controls.js and site/index.html, and the README is updated with the new step and controls.

User · 2026-10-14 18:03 UTC

The "L" just signifies left hemisphere and can be kept as is. Some samples were taken from left and right hemispheres, so the L is necessary to distinguish the two samples.

The sidebar is still pretty crowded and some functionalities a little unclear. Let's do the following for the *Spatial Layers* part to try and clean things up:

1. The DAPI dropdown to adjust B&C should initially be collapsed.
2. The DAPI and H&E alpha sliders should be a shorter slider, similar in size to the one for the Cell layer, so that it displays in-line with the label.
3. An on-hover hint for the little slider buttons on each alpha slider that says "Transparency: [current value]"
4. A small label next to the slider itself that says "Transparency" or "Alpha"

Then, for the Cell Sets bar:

1. The primary checkbox next to each cell set should be in line with the title, dropdown arrow, and vertical ellipses rather than on its own line.

For the Gene selection bar:

1. I currently have to shift click to select multiple genes. The user should be able to perform a single left click and select an additional gene by default.
2. The title should be changed to "Gene Expression (1 or Multiple)"

For the Display options bar, there is blank space (for the gene expression legend) that persists when gene expression is not being shown.

I do not see the Z score by sample or all data toggle in the interface. This should b e an option exposed to the user in the display options.

Finally, all four of these menus (Display Options, Spatial Layers, Cell Sets, Genes) should be collapsable.

Claude · 2026-10-14 18:04 UTC

This is a set of changes to Vitessce's own panels, so first I'll check what each one exposes. The things to find out:

- whether the DAPI channel controls start open
- how the gene list handles clicks
- what the layer controller's opacity sliders look like
- whether Vitessce has any built-in panel collapsing

▸ Bash — Check gene list click handling
▸ Bash — Find multi-select click logic in bundle
▸ Bash — Read SelectableTable implementation
▸ Bash — Inspect image layer controller expand state

The npm source for 4.0.8 shows the image-layer opacity slider inline, but the rendered page puts it on its own row. I'll compare against the bundle we actually serve:

▸ Bash — Locate image layer opacity slider in bundle
▸ Bash — Find layer row cell styles in bundle
▸ Bash — Read responsive layer row styles
▸ Bash — Read Vitessce view sizing hooks
▸ Bash — Confirm patch targets are unique in bundle
▸ Read — 04_vendor_vitessce.py
▸ Bash — Extend vendor script to four patches and re-vendor
▸ Bash — List views that accept closeButtonVisible
▸ Bash — Check layer controller and spatial close-button props
▸ Bash — Read layer controller subscriber props
▸ Bash — Set gene list title and hide close buttons
▸ browser:browser_batch — resize_window; navigate; computer; javascript_tool
▸ browser:javascript_tool — javascript_exec
▸ browser:javascript_tool — javascript_exec
▸ Bash — Restructure Display options panel; z-score toggle always shown

Now the page side: CSS grid layout, collapsible panels, the inline cell-set checkbox, and the Alpha labels with hover hints. I'll view the current style block before editing it:

▸ Read — site/index.html
▸ Bash — Add grid layout, collapse, alpha-label and checkbox CSS

Next I'll write `site/sidebar.js`. It tags the panels, sets the grid rows, handles collapsing (remembered per browser), and keeps the "Transparency: …" hover hints current:

▸ Write — site/sidebar.js
▸ browser:browser_batch — navigate; computer; read_console_messages; computer
▸ Bash — Add cache-busting version to module imports
▸ browser:browser_batch — navigate; computer; read_console_messages; computer
▸ browser:javascript_tool — javascript_exec
▸ Bash — Size grid from viewport height
▸ browser:browser_batch — navigate; computer; computer
▸ browser:computer — zoom
▸ browser:javascript_tool — javascript_exec
▸ Bash — Keep layer rows on one line
▸ browser:browser_batch — navigate; computer; computer; javascript_tool
▸ Bash — Fix layer name vertical alignment
▸ browser:browser_batch — navigate; computer; computer; javascript_tool
▸ Bash — Move Alpha label into slider cell padding
▸ browser:browser_batch — navigate; computer; computer; javascript_tool

The sliders are about 73 px now, each with its Alpha label inline. Next I'll drag the H&E slider to about the middle and check that the hint updates:

▸ browser:browser_batch — Sets the H&E alpha slider to about half; computer; javascript_tool; computer
▸ browser:browser_batch — computer; Sets the H&E alpha slider to about half; computer; javascript_tool; computer

The hint follows the value ("Transparency: 95% (alpha 0.05)" after the click, and the H&E dimmed accordingly). Next I'll collapse Cell Sets and confirm the gene list grows into the space:

▸ browser:browser_batch — Collapses the Cell Sets panel; computer; computer

Cell Sets collapsed to its title bar and the gene panel grew into the space. The gene list itself still shows only 8 rows, though. I'll check whether that's a stale capture or a real sizing problem:

▸ browser:javascript_tool — javascript_exec

That was a stale capture: the list is 375 px tall with 26 rows rendered. Next, single-click multi-select: click ABCC9, then ACHE, with no Shift.

‣ browser:browser_batch — Selects gene ABCC9; computer; Adds gene ACHE with a plain click; computer; computer

The first click selected ABCC9 and the legend appeared. That pushed the lower panels down, so my second click landed between two headers. I'll click ACHE at its new position:

‣ browser:browser_batch — Adds gene ACHE with a plain click; computer; computer
‣ browser:browser_batch — Removes gene ACHE with a plain click; computer; computer; find
‣ browser:browser_batch — Expands the DAPI channel controls; computer; computer
‣ browser:javascript_tool — javascript_exec
‣ Bash — Restrict layer-row CSS to row containers
‣ browser:browser_batch — navigate; computer; find
‣ browser:browser_batch — Expands the DAPI channel controls; computer; computer
‣ browser:browser_batch — resize_window; navigate; computer; Expands the Cell Sets panel; computer; computer
‣ browser:computer — screenshot

The light theme works too. Last, I'll expand Cell Sets again and check the inline checkboxes:

‣ browser:browser_batch — Expands the Cell Sets panel; computer; computer

The checkboxes are inline now but sit a few pixels low. I'll reset their container's spacing:

‣ Bash — Vertically align cell-set level checkboxes
‣ browser:browser_batch — navigate; computer; computer; computer

The checkboxes are aligned now. I'll reset the browser pane and update the README with the new files and patches:

‣ browser:resize_window
‣ Bash — Update README for sidebar changes and patches

All the sidebar changes are in and working at http://localhost:8765. I clicked through each one in the browser pane, in both dark and light themes.

Spatial Layers

- DAPI's brightness/contrast controls now start collapsed; the chevron on the DAPI row expands them.
- DAPI and H&E now have short inline sliders like the Cell row, with a small "Alpha" label beside each.
- Hovering a slider shows "Transparency: 20% (alpha 0.80)". I added the alpha number because the slider itself measures opacity (right = more opaque), so "Transparency: 0.80" alone would read backwards. Say if you'd prefer it worded differently.

Cell Sets

- Each grouping's checkbox is now on the same line as its name, arrow and ⋮ menu.

Gene Expression (1 or Multiple)

- Renamed as you asked.
- A plain click now adds a gene, and clicking a selected gene removes it; no Shift needed. Out of the box, Vitessce replaces the selection unless Shift is held, so this needed another small patch to the bundled copy.

Display options

- No more blank space: the panel is only as tall as its contents, and the legend and "Clear genes" button appear only while cells are coloured by gene.
- The z-score toggle ("Multi-gene z-score: this sample / all samples") is now always shown. You couldn't see it before because it only appeared once two genes were selected, and that took Shift-clicking.
- One side effect: because the panel sizes to its contents, the panels below shift down when the gene legend appears.

Collapsing

- Click any sidebar panel's title (▾/▸) to collapse it to its title bar; the remaining panels expand into the space. The state is remembered per browser.
- I removed Vitessce's × close buttons, since a closed panel can't be reopened without reloading.

To make this work: Vitessce has no setting for these, so I changed the bundled copy and the page itself.

- **Bundle patches:** there are now four, applied by `04_vendor_vitessce.py`: hide unselected cells, click to add genes, DAPI channel controls start collapsed, and inline opacity sliders.
- **Sidebar layout:** a new `site/sidebar.js` plus CSS in `site/index.html` re-lay out the sidebar for collapsing, and add the Alpha labels, hover hints and inline checkboxes.
- **When upgrading Vitessce:** these depend on Vitessce's internal page structure, so they'll need re-checking. The README now says so.
- **When editing the site's JS files:** bump `ASSET_VERSION` in `index.html`, otherwise browsers keep serving the old copies. That's what hid the first round of these changes during my testing.

About the "(L)": this sample's object has no hemisphere field. Its `brnum` and `sample_id` are both just "Br0942", so the label currently reads `Br0942_E4hom_M_62` with no L. Is the hemisphere stored somewhere I can read it, such as a colData column in the other samples, or in the file or folder names? If not, a small sample-to-hemisphere table would work. Either way I'll add it to the label (`Br0942L_E4hom_M_62`) and to the title.

User · 2026-10-15 01:39 UTC

Thanks! Could you put a little readme in the top-level directory that explains what changes were made to `04_vendor_vitessce.py` and any other files with a note that equivalent files might need updating if the package is updated in place?

My apologies regarding the confusion on the “L” in the brain numbers. If the sample was a left hemisphere, the `brnum` / `sample_id` columns follow the format `Br1234L`, otherwise no trailing letter is included.

Can you re-launch the viewer? It seems to have disconnected.

Claude · 2026-10-15 01:41 UTC

I’ll write the customisation notes first, then relaunch the viewer.

On the “L”: no change is needed. The label and title take `brnum` exactly as stored, so a left-hemisphere sample `Br1234L` will show up as `Br1234L_<APOE>_<Sex>_<age>` and “Br1234L (…)”.

- ▸ Write — VITESSCE_CUSTOMIZATIONS.md
- ▸ browser:preview_start — xenviewer-site

The server reports it’s still running, so I’ll reload the page and confirm it actually serves:

- ▸ browser:browser_batch — navigate; computer; read_console_messages; computer
- ▸ Bash — Give the sidebar a minimum width
- ▸ browser:browser_batch — navigate; computer; computer
- ▸ Bash — Make the title truncate with an ellipsis
- ▸ browser:browser_batch — navigate; computer; computer

I’ve written the notes and relaunched the viewer.

Customisation notes: VITESSCE_CUSTOMIZATIONS.md in the top-level folder covers every place that depends on Vitessce internals:

- the four patches in `04_vendor_vitessce.py`: what each changes, which upstream function it targets, and why
- the CSS and settings in `site/index.html`
- the page structure `site/sidebar.js` hooks into
- the plugin wiring in `site/xenviewer-controls.js`
- the config settings and package pins in `03_package_vitessce.py`

It opens with a warning that these may need updating if Vitessce (the JS bundle or the Python package) is updated or files are replaced in place. The patch script stops with an error when a patch target is missing, but the page-level code just stops working without any error, so the file ends with a 6-step re-test checklist.

The “L”: nothing needed to change. Labels and titles use `brnum` exactly as stored, so a left-hemisphere sample will show as `Br1234L_E4hom_M_62` and “Br1234L (APOE4 homozygous, …)”. Br0942 has no L, so it isn’t a left-hemisphere sample.

Viewer: the local server was still running; the page in the browser pane had just disconnected. It’s reloaded and working at http://localhost:8765.

The pane is only about 800 px wide right now, which showed two problems, and I fixed both:

- The sidebar squeezed down to about 260 px and clipped its controls. It now keeps a 340 px minimum, and the image pane shrinks instead.
- The centred title was cut off at both ends. It now truncates at the end with "…".

For comfortable use, widen the pane or open http://localhost:8765 in a normal browser window.

User · 2026-10-15 01:57 UTC

Alright, I think we're closing in on a good interface here. After this last little UI oddity, I propose we do the following:

1. Have you add a second sample to the dataset/viewer and make sure everything works on another sample
2. Once we clear that, I'll try adding 2 samples in a batched/parallel manner to test processing of multiple samples (and learn how to run the setup process)

But first, there's a couple small, strange issues persisting in the UI–a tab or different justification for "Cell" (lefthand red box) and different picture icons for the H&E and DAPI selectors (red/orange square; compare to green for desired).

Let's also add a second slider to the display options panel that allows the user to thicken/thin the domain boundary polygon as desired.

The legend is also a little on the small side, so finally let's add a little "legend fonts" dropdown so that the user can make these more visible if need be.

Finally, I note that the bottom says "21,923 spots, 2 images", but we would term these "cells" rather than spots in Xenium data. Don't bother with fixing that if it involves editing more of Vitessce's shipped code though.

Doy. I did not send the screenshot illustrating the UI issues.

[screenshot attached]

Claude · 2026-10-15 02:01 UTC

I'll look at the DOM behind each of these before changing anything: the Cell row's empty box, the two image-icon sizes, the label alignment, the inset legend's container, and the "spots" footer.

▸ browser:browser_batch — resize_window; navigate; computer; javascript_tool
▸ browser:javascript_tool — javascript_exec

The panels aren't being tagged, so `sidebar.js` isn't running. Checking for an error:

▸ browser:javascript_tool — javascript_exec
▸ Bash — Use setTimeout for sidebar layout pass
▸ Bash — Bump asset version
▸ browser:browser_batch — navigate; computer; javascript_tool

Every element measures 0×0, so the Layers panel is probably collapsed (state remembered from my earlier testing). A screenshot will confirm:

▸ browser:computer — screenshot

Spatial Layers was collapsed (state remembered from earlier). Expanding it and measuring again:

▸ browser:browser_batch — Expands the Spatial Layers panel; computer; javascript_tool
▸ Bash — Locate domain packaging code to change
▸ Read — 03_package_vitessce.py
▸ Read — 03_package_vitessce.py

Now I'll refactor the packager so the domain masks are written one channel at a time. Seven widths × six domains is 42 full-resolution channels, too much to hold in memory at once.

▸ Bash — Write domain masks at multiple line widths
▸ Bash — Add boundary width slider and legend size dropdown

Now the page-side CSS: hide the colour-picker cell, centre the names, fix the icon sizes, and scale the legends. Plus the "spots" → "cells" footer fix in `sidebar.js`:

▸ Bash — Apply CSS fixes, footer relabel; repackage with widths

Domain masks at all seven widths add 23 MB per sample, and packaging still takes about 14 s. I'll test in the browser:

▸ browser:browser_batch — navigate; computer; read_console_messages; computer
▸ browser:find — LC boundary line width
▸ browser:find — Legend size
▸ browser:browser_batch — Sets the LC boundary width to 20 µm; Sets the legend size to Large; computer; computer
▸ browser:find — combobox
▸ browser:browser_batch — Sets the LC boundary width to 20 µm; Sets the legend size to Large; computer; computer

The Large legend works. At the whole-section view, 20 µm is only about 1–2 screen pixels, so I'll zoom in to compare 20 µm against 1 µm:

▸ browser:browser_batch — computer; computer; computer; computer; Sets the LC boundary width to 1 µm; computer; computer

At 20 µm no outline appeared at all, while 1 µm draws fine. That could be tiles not yet loaded when I captured, so I'll set 20 µm again and wait longer:

▸ browser:browser_batch — Sets the LC boundary width to 20 μm; computer; computer; javascript_tool
▸ Bash — Inspect segmentation channel selection logic
▸ Bash — Read bitmask layer creation code
▸ Bash — Read segmentation selection code

The selection code passes `spatialTargetC` through `getChannelIndex(...)`. I'll check what that function does with a numeric index:

▸ Bash — Read getChannelIndex implementation
▸ Bash — Test: drop obsTypesFromChannelNames from config
▸ browser:browser_batch — navigate; computer; computer; computer; computer; find
▸ browser:browser_batch — computer; computer; Sets the LC boundary width to 20 μm; computer; computer; javascript_tool

Channel 6 is still never requested. Before going further, I'll confirm the slider change actually reached Vitessce's state, since the label is derived from it:

▸ browser:javascript_tool — javascript_exec
▸ browser:browser_batch — Sets the LC boundary width to 12 μm; computer; Sets the LC boundary width to 8 μm; computer; computer; javascript_tool

---

User · 2026-10-15 02:06 UTC

(sent while Claude was working)

How about we just try setting it to something like 8μm

Claude · 2026-10-15 02:06 UTC

Testing 8 μm is the batch that's running now; I'll report what it shows as soon as it finishes.

User · 2026-10-15 02:06 UTC

[Request interrupted by user]

That was unclear. How about we just used a fixed setting of (actually 7) μm thickness, instead of the current 1μm, and see how that looks?

Claude · 2026-10-15 02:10 UTC

Understood: I'll drop the slider and draw every outline at a fixed 7 μm. That also sidesteps the channel-switching problem: even switching to the 8 μm channel (number 4) didn't make Vitessce load it, so the 42-channel approach wasn't working anyway. It goes back to one channel per domain, the setup the domain dropdown already used successfully.

▸ Bash — Revert to fixed 7 μm outlines, one channel per domain

Repackaged with 7 μm outlines (3.3 MB of masks instead of 23 MB). Next I'll check it in the browser at whole-section and zoomed views, and confirm the dropdown still switches domains:

- ▸ browser:browser_batch — resize_window; navigate; computer; computer
- ▸ Write — serve.py
- ▸ Bash — Point preview launch config at no-cache server
- ▸ browser:preview_list
- ▸ browser:preview_stop
- ▸ browser:preview_start — xenviewer-site

The no-cache server is running. Some files may still be cached in the browser from before, so I'll reload and check whether the domain metadata came from the network or the cache:

- ▸ browser:browser_batch — resize_window; navigate; computer; javascript_tool; computer
- ▸ Bash — Switch to no-cache and list zarr metadata files
- ▸ browser:preview_stop
- ▸ browser:preview_start — xenviewer-site
- ▸ Bash — Build list of files to refresh in browser cache
- ▸ browser:javascript_tool — javascript_exec
- ▸ browser:browser_batch — navigate; computer; computer; javascript_tool

The metadata is now fresh (304 revalidations), but the outline still doesn't show. Checking which domain chunks are being requested:

- ▸ browser:javascript_tool — javascript_exec

The right chunks are loading (channel 0 at the expected level), and the metadata shows the new 6-channel shape. I'll check the data itself in Python and take a zoomed look in the viewer:

- ▸ Bash — Verify domain mask contents
- ▸ browser:browser_batch — computer; computer; computer; computer
- ▸ browser:browser_batch — navigate; computer; computer

The outline still doesn't show at the initial whole-section zoom, even though zoomed-in views are fine. At this zoom Vitessce uses the coarsest pyramid level, so I'll check that level's data:

- ▸ Bash — Check coarsest domain pyramid level

The coarsest level contains the outline (963 pixels), so the data is fine. It may be a rendering or capture issue with the hidden pane. I'll nudge the view so it re-renders:

- ▸ browser:browser_batch — computer; computer; computer; computer; computer; computer; tabs_context

▸ browser:resize_window
▸ Bash — Document 7 µm outlines, caching and serve.py

The interface fixes are in, and the LC boundary outlines are now a fixed 7 µm thick. The viewer is running at http://localhost:8765.

What changed:

- **Cell row:** the empty box was a colour-picker cell that only the Cell row has. Cells are always coloured by cluster or gene, so I've hidden it, and Cell, DAPI and H&E now line up. The layer names are also vertically centred against the eye icons.
- **Picture icons:** Vitessce sizes these as a percentage of their cell, and H&E's cell is narrower, so its icon came out half-size. All layer icons are now a fixed 22 px.
- **Legend size:** a new dropdown in Display options (Small / Medium / Large / Extra large). It scales both Vitessce's legend inside the image and the gene legend in the panel, and is remembered per browser.
- **"spots" → "cells":** the footer now reads "21,923 cells, 2 images". This is page code only; Vitessce's shipped files are unchanged.
- **Boundary width:** I removed the slider as you suggested. At 7 µm the outline reads clearly at whole-section zoom and still doesn't cover the edge cells when zoomed in.

What actually broke the slider: the browser was caching Vitessce's image-data description files. Python's built-in web server doesn't tell browsers not to cache, so after I re-packaged the outlines (6 → 42 image layers), the browser kept the old 6-layer description and never requested the thicker-line layers. To fix and prevent this:

- The local preview now runs through a new `serve.py`, which tells browsers to re-check every file on each load. Unchanged files are confirmed with a tiny "not modified" reply, so it stays fast.
- The README now says your institute server needs the same setting (`Cache-Control: no-cache`) on `site/data/`. Otherwise people could see broken data after you update a sample.
- The slider would probably work now that caching is fixed. If you want variable thickness back, I can restore it.

Next step: a second sample. Can you place its files somewhere for me, or tell me where they are? I need the same three as for Br0942:

- the annotated SpatialExperiment `.RDS`
- `morphology_focus_0000.ome.tif`
- the H&E stored inside the SPE object

A left-hemisphere sample (`Br####L`) would also let us check the "L" labelling.

User · 2026-10-15 16:10 UTC

Very quick simple question: max pixel dimension for each level of the Br0942 DAPI pyramid

---

Claude · 2026-10-15 16:11 UTC

▸ Bash — Read DAPI pyramid level shapes

The largest dimension of each level of the packaged Br0942 DAPI pyramid is its height, in pixels:

Level	Height × width (px)	Max dimension (px)	µm per pixel
0 (full resolution)	24019 × 22913	**24019**	0.2125
1	12009 × 11456	**12009**	0.425
2	6004 × 5728	**6004**	0.85
3	3002 × 2864	**3002**	1.7
4	1501 × 1432	**1501**	3.4
5	750 × 716	**750**	6.8
6	375 × 358	**375**	13.6

These are the same seven levels as in Xenium's original `morphology_focus_0000.ome.tif`; the packaging step re-saves them in a browser-readable format without resizing.

User · 2026-10-15 16:44 UTC

Alright, I've added a folder for the second sample, `Br1039L`. The samplewise directories and the starting SPE filenames reflect the naming conventions that will be used for the full dataset build.

Additionally, the full dataset build will use the CRAN pkg `qs2` rather than read/writeRDS, as qs2 is much faster (which may be necessary if I decide to use a higher-resolution H&E image).

Assuming the one-sample SPE and the max intens prjxn DAPI are all you need, go ahead and give this sample a shot!

Claude · 2026-10-15 16:44 UTC

- ▸ Bash — Inspect second sample folder and check qs2
- ▸ Bash — Compare Br1039L object with Br0942
- ▸ Bash — Find SPE readers and hardcoded paths
- ▸ Bash — Read inspect script header
- ▸ Bash — Make inspect script per-sample; add qs2 reading

Both R scripts now read `.qs2` or `.RDS`. Next I'll run the inspection and registration check on Br1039L, then the R prep step:

- ▸ Bash — Run inspection and alignment check on Br1039L

Registration looks good. The best shift is 1 low-res H&E pixel (about 6 µm), and it improves the overlap score only from 0.859 to 0.861, which is within noise. I'll check the overlay image:

- ▸ Read — viewer_data/Br1039L/inspect/01_alignment_he_vs_dapi.png

The tissue edges and the tear on the left line up with the DAPI nuclei. Next I'll run the R prep, packaging, and global z-scores (now pooled over both samples):

- ▸ Bash — Run R prep for Br1039L
- ▸ Read — viewer_data/Br1039L/qc_domains.png

The outlines look right: the LC sits along the left edge, the tear inside it stays as a hole, and the satellite patches with 10+ cells are kept. Next, packaging and the pooled z-scores:

- ▸ Bash — Package Br1039L and recompute global z-scores

Both samples are packaged. Br1039L took 8 s, and the across-samples z-scores are now pooled over 35,682 cells from both. I'll check Br1039L in the viewer:

- ▸ browser:browser_batch — resize_window; navigate; computer; read_console_messages; computer; computer; computer; computer
- ▸ browser:preview_start — xenviewer-site
- ▸ browser:browser_batch — resize_window; navigate; computer; read_console_messages; computer; computer; computer; computer
- ▸ browser:browser_batch — Focuses the gene search box; Types TH into the gene search box; computer; Selects gene TH; computer; computer

After selecting TH, the image went black and the legend is stuck on "…". This might be the hidden-pane rendering pause again, or a real error. Checking:

- ▸ browser:browser_batch — computer; read_console_messages; javascript_tool; computer; computer; computer

That was the paused hidden pane again. After a nudge, TH (0–5.84) is drawn, with coloured cells inside the outline. Next I'll add DBH and compare the "this sample" and "all samples" ranges:

- ▸ browser:browser_batch — Selects the text in the gene search box; Types DBH into the gene search box; computer; Adds gene DBH to the selection; computer; javascript_tool; find
- ▸ browser:browser_batch — Switches z-scoring to all samples; computer; javascript_tool; computer; computer; computer
- ▸ Bash — Independently compute expected legend ranges
- ▸ browser:browser_batch — find
- ▸ browser:browser_batch — Clears the gene selection; computer; computer; Turns on the DAPI layer; Turns off the H&E layer; computer; computer; computer

The panels shifted up when the gene legend disappeared, so my clicks missed the layer toggles. Retrying at the new positions:

▸ browser:browser_batch — Turns on the DAPI layer; Turns off the H&E layer; computer; computer; computer
▸ browser:resize_window
▸ Bash — Update README for qs2 and per-sample inspection

Br1039L is in the viewer and everything I tested works. Choose `Br1039L_E4hom_F_51` from the sample picker at http://localhost:8765.

The SPE (`.qs2`) and the DAPI focus image were all it needed:

Step	Time	Result
Inspection and registration check	6 s	H&E lines up with DAPI. The best shift is about 6 µm (one low-res H&E pixel) and barely improves the fit, so it's noise. Plots are in `viewer_data/Br1039L/inspect/`.
R prep	7 s	13,759 cells. All 6 LC domains outlined; the tissue tear inside the LC stays as a hole.
Packaging	8 s	352 MB in `site/data/Br1039L/`
Z-scores across samples	–	Now pooled over both samples (35,682 cells)

What I checked in the viewer for Br1039L:

- the label and title, including the "L"
- H&E and DAPI each line up with the cells; the LCNE cells fill the LC outline
- single-gene colouring (TH: 0–5.84 logcounts)
- two-gene mean z-scores: TH+DBH reads −0.37 to 4.64 within this sample and −0.35 to 5.35 across both samples. Both match an independent Python calculation exactly.

Changes needed for the full build:

- **Reading `.qs2`:** both R scripts now read `.qs2` via `qs2::qs_read` (or `.RDS`, chosen by file extension).
- **Inspection script:** `01_inspect_inputs.R` moved from `Br0942/` to the top level and now takes a sample's files as arguments, so the registration check can run on every sample. Br0942's earlier outputs are still in `Br0942/`.
- **Two new snRNA-seq labels:** `excitatory_23` and `neurons_ambiguous_18` weren't in the palette file, so they got automatic colours and are now saved there. There's no paper palette for the snRNA-seq labels, so if you want to choose these colours yourself, now is a good time.

For your batch test, each sample takes three commands, with one step at the end:

```
Rscript 01_inspect_inputs.R Br1039L/Br1039L.qs2 Br1039L/morphology_focus_0000.ome.tif
viewer_data/Br1039L/inspect
```

```
Rscript 02_prep_viewer_data.R Br1039L/Br1039L.qs2 Br1039L/morphology_focus_0000.ome.tif
viewer_data
```

```
uv run python 03_package_vitessce.py viewer_data/Br1039L
```

Then, once all samples are packaged:

```
uv run python 05_global_zscores.py --workers 8
```

The per-sample steps are independent, so several samples can run in parallel. The one catch is that each packaging run also edits the shared `site/samples.json`, so two runs finishing at the same moment could overwrite each other's entry. A small batch driver that runs 8 samples at a time and writes `samples.json` once at the end would avoid that. Shall I write it before your two-sample test?

User · 2026-10-31 00:06 UTC

Please export this entire chat to a simple markdown file, then saved as PDF, before we continue.

Claude · 2026-10-31 00:06 UTC

I'll use the app's transcript export rather than retyping the conversation, then convert that to PDF.

▸ ToolSearch — select:mcp__ccd_session_mgmt__export_transcript